

  astro-datalab / notebooks-latest

Q Type  to search 

+        Security and quality  Insights

 master [notebooks-latest](#) / [04_HowTos](#) / [SPARCL](#)
/ [Plot_SPARCL_Spectra_with_Jdaviz.ipynb](#) 

Q Go to file  T 

 **weaverba137** update dates on two notebooks ✓

6c6a7fe · last month  History

 master [notebooks-latest](#) / [04_HowTos](#) / [SPARCL](#) / [Plot_SPARCL_Spectra_with_Jdaviz.ipynb](#) 

Preview Code Blame   Raw   

SPARCL + Jdaviz

Table of contents

- [Goals](#)
- [Introduction](#)
- [Disclaimer & attribution](#)
- [Imports and setup](#)
- [Search for spectra in SPARCL](#)
- [Retrieve spectra from SPARCL and prepare plot data](#)
- [Plot spectra using Jdaviz](#)

Goals

- Search for and obtain spectra from DESI DR1 using the [NOIRLab SPARCL service](#)
- Plot spectra using the [Jdaviz](#) data analysis visualization tool

Introduction

This notebook demonstrates how to find and retrieve spectroscopic data for certain objects from the DESI DR1 data set using [SPARCL](#) (Spectra Analysis and Retrievable Catalog Lab) and display an interactive plot of an object's spectrum using [Jdaviz](#).

This notebook is based on [an example](#) originally developed by [Camilla Pacifici](#). Additional assistance was provided by [Brett Morris](#).

Packages:

- [SPARCL](#)
- [Jdaviz](#)
- [jupyterlab-sidecar](#)

Disclaimer & attribution

Disclaimers

Note that using the Astro Data Lab constitutes your agreement with our minimal [Disclaimers](#).

Acknowledgments

If you use **Astro Data Lab** in your published research, please include the text in your paper's Acknowledgments section:

This research uses services or data provided by the Astro Data Lab, which is part of the Community Science and Data Center (CSDC) Program of NSF NOIRLab. NOIRLab is operated by the Association of Universities for Research in Astronomy (AURA), Inc. under a cooperative agreement with the U.S. National Science Foundation.

If you use **SPARCL jointly with the Astro Data Lab platform** (via JupyterLab, command-line, or web interface) in your published research, please include this text below in your paper's Acknowledgments section:

This research uses services or data provided by the SPectra Analysis and Retrievable Catalog Lab (SPARCL) and the Astro Data Lab, which are both part of the Community Science and Data Center (CSDC) Program of NSF NOIRLab. NOIRLab is operated by the Association of Universities for Research in Astronomy (AURA), Inc. under a cooperative agreement with the U.S. National Science Foundation.

In either case **please cite the following papers**:

- Data Lab concept paper: Fitzpatrick et al., "The NOAO Data Laboratory: a conceptual overview", SPIE, 9149, 2014, <https://doi.org/10.1117/12.2057445>
- Astro Data Lab overview: Nikutta et al., "Data Lab - A Community Science Platform", Astronomy and Computing, 33, 2020, <https://doi.org/10.1016/j.ascom.2020.100411>

If you are referring to the Data Lab JupyterLab / Jupyter Notebooks, cite:

- Juneau et al., "Jupyter-Enabled Astrophysical Analysis Using Data-Proximate Computing Platforms", CiSE, 23, 15, 2021, <https://doi.org/10.1109/MCSE.2021.3057097>

If publishing in a AAS journal, also add the keyword: `\facility{Astro Data Lab}`

And if you are using SPARCL, please also add `\software{SPARCL}` and cite:

- Juneau et al., "SPARCL: SPectra Analysis and Retrievable Catalog Lab", Conference Proceedings for ADASS XXXIII, 2024 <https://doi.org/10.48550/arXiv.2401.05576>

The NOIRLab Library maintains [lists of proper acknowledgments](#) to use when publishing papers using the Lab's facilities, data, or services.

For this notebook specifically, please acknowledge:

- Jdavis: https://jdavis.readthedocs.io/en/latest/index_citation.html

Imports and setup

```
In [ ]: import os
```

```

import sys
# Handle warnings below
import warnings
# Numpy import
import numpy as np
# SPARCL import
from sparcl import __version__ as sparcl_version
from sparcl.client import SparclClient
# Jdaviz import
from jdaviz import __version__ as jdaviz_version
from jdaviz import Specviz
# Jupyter
import ipywidgets as widgets
from IPython.display import display
# Specutils import
from specutils import __version__ as specutils_version
try:
    from specutils import Spectrum
except:
    from specutils import Spectrum1D as Spectrum
# Astropy import
import astropy.units as u
from astropy.nddata import InverseVariance, StdDevUncertainty
# Jdaviz can emit warnings about Angstrom which we will ignore.
warnings.filterwarnings('ignore', category=u.UnitsWarning, message=r'*.V0Unit standard.*')
print(f"sparcl_version=={sparcl_version}")
print(f"jdaviz_version=={jdaviz_version}")
print(f"specutils_version=={specutils_version}")

```

Set up the SPARCL client

```

In [ ]: client = SparclClient()
        client

```

View the full list of fields for the DESI DR1 data set that can be obtained from SPARCL

```

In [ ]: desi_fields = client.get_all_fields(dataset_list=['DESI-DR1'])
        print(desi_fields)

```

Search for spectra in SPARCL

Using the `client.find()` method, we obtain the following fields from the DESI DR1 data set using SPARCL:

- `sparcl_id` : Universally Unique Identifier for spectrum in SPARCL
- `ra` : Right Ascension in degrees
- `dec` : Declination in degrees
- `spectype` : Spectral type of the object (STAR, GALAXY, or QSO)
- `subtype` : Spectral subtype
- `specid` : Dataset-specific spectrum identifier (may not be unique)
- `redshift` : Measured redshift
- `redshift_err` : Uncertainty on the measured redshift

And apply the following constraints:

- `2.0 < redshift < 5.0`
- `redshift_warning = 0` (Redshift warning bitmask measured by Redrock)
- `specprimary = 1` (Boolean flag (True/False) for the primary coadded spectrum for a given target object)
- `spectype = QSO`

- data_release = DESI DR1

```
In [ ]: outfields = ['sparcl_id', 'ra', 'dec', 'spectype', 'subtype', 'specid',
                  'redshift', 'redshift_err'],
constraints = {'redshift': [2.0, 5.0],
              'redshift_warning': [0],
              'specprimary': [1],
              'spectype': ['QSO'],
              'data_release': ['DESI-DR1'],
              }
```

```
In [ ]: found = client.find(outfields=outfields,
                           constraints=constraints,
                           limit=50)
```

DESINAME

The function below is copied from [desiutil.names](#). DESINAME is a convenient RA, Dec-based label that is consistent with IAU and NED standards.

Although as of DESI DR1, DESINAME is available from the SPARCL database, it is not a CORE field, so `client.find()` will not return it. However, we need a label for these spectra to pass to `Specviz`, *before* the spectra are retrieved. So we will compute DESINAME as a work-around.

```
In [ ]: def radecc_to_desiname(target_ra, target_dec):
        """Convert the right ascension and declination of a DESI target
        into the corresponding "DESINAME" for reference in publications.
        Length of target_ra and target_dec must be the same if providing an
        array or list. Note that these names are not unique for roughly
        one percent of DESI targets, so also including TARGETID in
        publications is highly recommended for uniqueness.

        Parameters
        -----
        target_ra: array of :class:`float64`
            Right ascension in degrees of target object(s). Can be float, double,
            or array/list of floats or doubles
        target_dec: array of :class:`float64`
            Declination in degrees of target object(s). Can be float, double,
            or array/list of floats or doubles

        Returns
        -----
        array of :class:`str`
            The DESI names referring to the input target RA and DEC's. Array is
            the same length as the input arrays.

        """
        # Convert to numpy array in case inputs are scalars or lists
        target_ra, target_dec = np.atleast_1d(target_ra), np.atleast_1d(target_dec)

        # Number of decimal places in final naming convention
        precision = 4

        # Truncate decimals to the given precision
        ratrunc = np.trunc((10 ** precision) * target_ra).astype(int).astype(str)
        dectrunc = np.trunc((10 ** precision) * target_dec).astype(int).astype(str)

        # Loop over input values and create DESINAME as: DESI JXXX.XXXX+/-YY.YYYY
        # Here J refers to J2000, which isn't strictly correct but is the closest
        # IAU compliant term
        desinames = []
        for ra, dec in zip(ratrunc, dectrunc):
            desiname = 'DESI J' + ra[:precision].zfill(3) + '.' + ra[precision:]
            # Positive numbers need an explicit "+" while negative numbers
```

```

# ...these numbers need an explicit + to make negative numbers
# already have a "-".
# zfill works properly with '-' but counts it in number of characters
# so need one more
if dec.startswith('-'):
    desiname += dec[:precision].zfill(3) + '.' + dec[-precision:]
else:
    desiname += '+' + dec[:precision].zfill(2) + '.' + dec[-precision:]
desinames.append(desiname)

return np.array(desinames)

```

Create DESINAME from the RA and Dec of each record:

```

In [ ]: for record in found.records:
        if record['_dr'] == 'DESI-DR1':
            record['DESINAME'] = radec_to_desiname(record['ra'], record['dec']).tolist()[0]
metadata = sorted(found.records, key=lambda x: x['DESINAME'])
ids = [m['sparcl_id'] for m in metadata]

```

Retrieve spectra from SPARCL and prepare plot data

Here we define a function that:

1. Uses the `client.retrieve()` method from SPARCL to retrieve spectra for the list of IDs from DESI DR1.
2. Creates a `specutils.Spectrum` object from the retrieved data.
3. Loads the spectrum, metadata, and model.
4. Caches the converted `Spectrum` object.

```

In [ ]: def on_selected(b):
        """Callback function to enable switching between different DESI spectra.

        Parameters
        -----
        b : object
            A callback object supplied by Specviz.
        """
        specunit = u.Unit('10-17 erg cm-2 s-1 AA-1')
        key = entries[b['new']][0]
        result = client.retrieve(uuid_list=[ids[b['new']]],
                                include=['data_release',
                                         'specid',
                                         'survey',
                                         'ra',
                                         'dec',
                                         'redshift',
                                         'redshift_err',
                                         'flux',
                                         'ivar',
                                         'wavelength',
                                         'mask',
                                         'model'])

        if key not in spectra:
            #
            # Note how the data and the model are prepared as two separate objects.
            #
            result_specutils = result.to_specutils()
            spectra[key] = (result_specutils,
                           Spectrum(spectral_axis=result_specutils.spectral_axis,
                                   flux=result_specutils.meta['model']*specunit))

            #
            # This block is retained for possible use with Jdaviz 4.
            #
            # result_specutils.flux = result_specutils.flux[0, :]
            # result_specutils.uncertainty = result_specutils.uncertainty.reshape((StdDevUncertainty) 0)

```

```

# result_specutils.uncertainty = result_specutils.uncertainty.represent_as(StdevUncertainty)[0],
# result_specutils.mask = result_specutils.mask[0, :]
# spectra[key] = (Spectrum(spectral_axis=result.records[0]['wavelength']*u.AA,
#                           flux=result.records[0]['flux']*specunit,
#                           uncertainty=InverseVariance(result.records[0]['ivar']*(specunit**-2)).
#                           mask=result.records[0]['mask'],
#                           redshift=result.records[0]['redshift']),
#                 Spectrum(spectral_axis=result.records[0]['wavelength']*u.AA,
#                           flux=result.records[0]['model']*specunit))
# print(spectra[key][0].flux.shape)
if 'old' in b:
#
# Remove the previous spectrum.
#
data0 = specviz.app.data_collection[0]
data1 = specviz.app.data_collection[1]
specviz.app.data_collection.remove(data0)
specviz.app.data_collection.remove(data1)

#
# Load the spectrum.
#
specviz.load_data(spectra[key][0], data_label=key + ' Data')
opt = specviz.plugins['Plot Options']
# Jdavis 3
opt.layer = key + ' Data'
# Jdavis 4
# opt.layer = key + ' Data [0]'
opt.line_color.value = '#000000' # Black
opt.line_as_steps = True
opt.uncertainty_visible = True
#
# Add metadata.
#
meta0 = specviz.app.data_collection[0].meta
meta0['RA'] = result.records[0]['ra']
meta0['Dec'] = result.records[0]['dec']
meta0['redshift'] = result.records[0]['redshift']
meta0['redshift uncertainty'] = result.records[0]['redshift_err']
#
# Load the model.
#
specviz.load_data(spectra[key][1], data_label=key + ' Model')
opt = specviz.plugins['Plot Options']
# Jdavis 3
opt.layer = key + ' Model'
# Jdavis 4
# opt.layer = key + ' Model [0]'
opt.line_color.value = '#FF0000' # Red
opt.line_as_steps = True
#
# This is a workaround for a bug where metadata is not initially displayed.
#
m = specviz.plugins['Metadata']
# Jdavis 3
m.dataset = key + ' Model'
m.dataset = key + ' Data'
# Jdavis 4
# m.dataset = key + ' Model [0]'
# m.dataset = key + ' Data [0]'

```

Plot spectra using Jdavis

Run the cell below to open the interactive plot in a new panel on the right. Information on the Toolbar options can be found in the [Jdavis User Guide](#).

This cell will also display a drop-down menu that controls the spectrum that will be displayed in the interactive plot.

```
In [ ]: #  
        # Set up Specviz and the drop-down menu.  
        #  
        spectra = dict() # This will cache the retrieved spectra.  
        entries = [(m['DESNAME'], i) for i, m in enumerate(metadata)]  
        select = widgets.Dropdown(options=entries, value=0, description='Spectrum:',)
```